ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Η/Υ

Εξάμηνο 8ο: Ψηφιακά VLSI

ΟΜΑΔΑ 23 - ΑΝΑΦΟΡΑ LAB1

Όνομα	Επώνυμο	Α.Μ.
Μιχαήλ	Παναγόπουλος	03122061
Αθανάσιος	Φειδάκης	03122676

Μέρος Α

➢ A.2 – dec8 (behavioral)

Κώδικας VHDL – Source Code

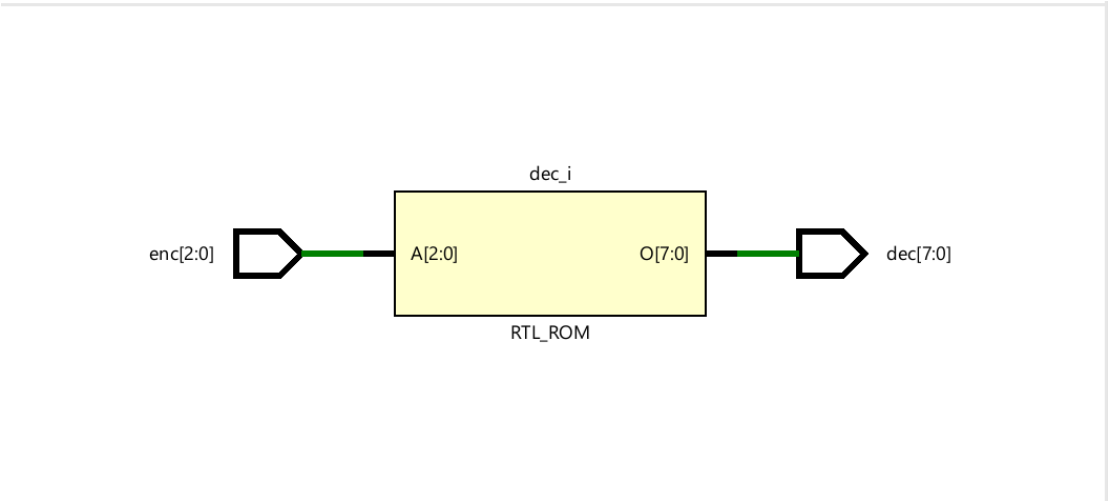
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;


entity decoder3_to_8_beh is
    port (
        enc         : in    std_logic_vector(2 downto 0);
        dec         : out   std_logic_vector(7 downto 0)
    );
end entity;

architecture behavioral_arch of decoder3_to_8_beh is

begin
    my_process: process (enc)
    begin
        case enc is
        when "000" =>
            dec <= "00000001";
        when "001" =>
            dec <= "00000010";
        when "010" =>
            dec <= "00000100";
        when "011" =>
            dec <= "00001000";
        when "100" =>
            dec <= "00010000";
        when "101" =>
            dec <= "00100000";
        when "110" =>
            dec <= "01000000";
        when "111" =>
            dec <= "10000000";
        when others =>
            dec <= "00000000";
        end case;

    end process my_process;


end behavioral_arch;

```

Κώδικας VHDL – Testbench

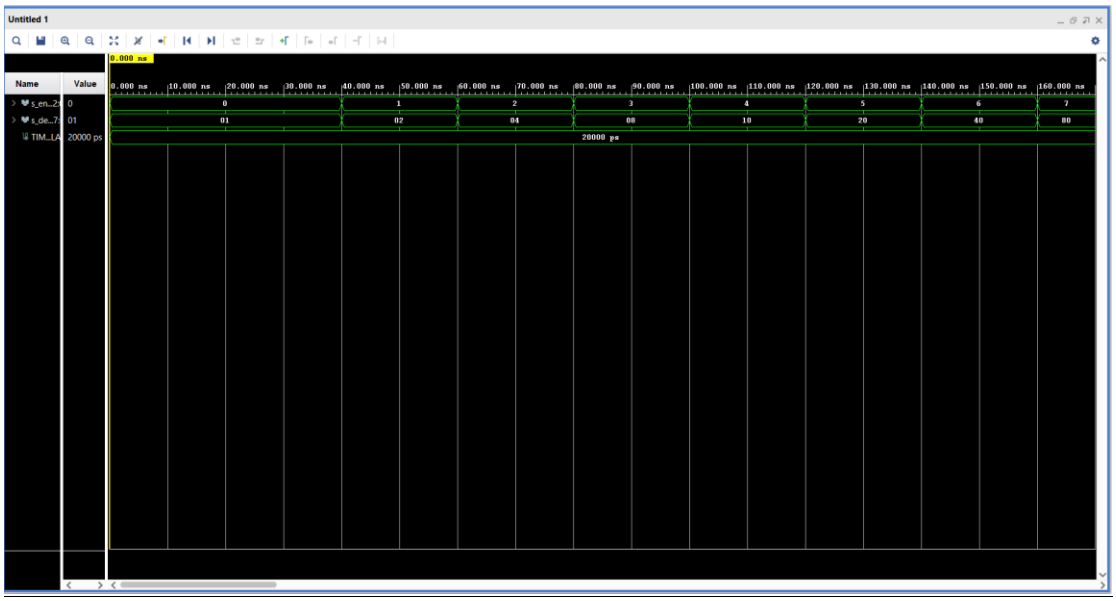
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity decoder3_to_8_beh_tb is
end entity;

architecture tb of decoder3_to_8_beh_tb is
    ------------------------------------------------------------
```

```
    -- COMPONENT
    ----------------------------------------------------------------
    component decoder3_to_8_beh is
        port (
            enc : in  std_logic_vector(2 downto 0);
            dec : out std_logic_vector(7 downto 0)
        );
    end component;

    ---------------------------------------------------------------
    -- SIGNALS
    ---------------------------------------------------------------
    signal s_enc : std_logic_vector(2 downto 0) := (others => '0');
    signal s_dec : std_logic_vector(7 downto 0);

    ---------------------------------------------------------------
    -- CONSTANTS
    ---------------------------------------------------------------
    constant TIME_DELAY : time := 20 ns;

begin

    DUT : decoder3_to_8_beh
        port map (
            enc => s_enc,
            dec => s_dec
        );


    STIMULUS : process
    begin

        -- INITIALIZE SIGNALS
        s_enc <= (others => '0');
        wait for (1*TIME_DELAY);

        for i in 0 to 7 loop
            s_enc <= std_logic_vector(to_unsigned(i, 3));
            wait for (1*TIME_DELAY);
        end loop;

        s_enc <= (others => '0');
        wait for (1 * TIME_DELAY);

        wait;

    end process;


end architecture;

```

RTL ANALYSIS – Schematic



SIMULATION



➢ A.2 – dec8 (Data-flow)

Κώδικας VHDL – Source Code

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;


entity decoder3_to_8_dataflow is
    port (
        enc         : in    std_logic_vector(2 downto 0);
        dec         : out   std_logic_vector(7 downto 0)
    );
```

```
end entity;

architecture dataflow_arch of decoder3_to_8_dataflow is

begin

    with enc select
        dec <=  "00000001" when "000",
                "00000010" when "001",
                "00000100" when "010",
                "00001000" when "011",
                "00010000" when "100",
                "00100000" when "101",
                "01000000" when "110",
                "10000000" when "111",
                "00000000" when others;


end dataflow_arch;

```

Κώδικας VHDL – Testbench

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity decoder3_to_8_dataflow_tb is
end entity;

architecture tb of decoder3_to_8_dataflow_tb is
    ---------------------------------------------------------------
    -- COMPONENT
    ----------------------------------------------------------------
    component decoder3_to_8_dataflow is
        port (
            enc : in  std_logic_vector(2 downto 0);
            dec : out std_logic_vector(7 downto 0)
        );
    end component;

    ---------------------------------------------------------------
    -- SIGNALS
    ---------------------------------------------------------------
    signal s_enc : std_logic_vector(2 downto 0) := (others => '0');
    signal s_dec : std_logic_vector(7 downto 0);

    ---------------------------------------------------------------
    -- CONSTANTS
    ---------------------------------------------------------------
    constant TIME_DELAY : time := 20 ns;

begin

    DUT : decoder3_to_8_dataflow
        port map (
            enc => s_enc,
            dec => s_dec
        );


    STIMULUS : process
    begin

        -- INITIALIZE SIGNALS
        s_enc <= (others => '0');
```

```
        wait for (1*TIME_DELAY);

        for i in 0 to 7 loop
            s_enc <= std_logic_vector(to_unsigned(i, 3));
            wait for (1*TIME_DELAY);
        end loop;

        s_enc <= (others => '0');
        wait for (1 * TIME_DELAY);

        wait;

    end process;


end architecture;

```

Σύντομη Περιγραφή Υλοποίησης

Τα αρχεία υλοποιούν έναν αποκωδικοποιητή 3 σε 8 (3-to-8 decoder), ο οποίος ενεργοποιεί μία από τις οκτώ εξόδους ανάλογα με την τιμή του 3-bit σήματος εισόδου enc. Η Behavioral μέθοδος (αρχείο _beh.vhd) χρησιμοποιεί ένα process και τη δομή ελέγχου case για να περιγράψει την αλγοριθμική συμπεριφορά του κυκλώματος , ενώ η Dataflow μέθοδος (αρχείο _dataflow.vhd) χρησιμοποιεί την ταυτόχρονη εντολή επιλογής with-select, περιγράφοντας απευθείας τη συνεχή ροή των δεδομένων προς την έξοδο. Η βασική τους διαφορά έγκειται στο επίπεδο αφαίρεσης: η Behavioral εστιάζει στη λογική σειρά εκτέλεσης εντός ενός block, ενώ η Dataflow στην παράλληλη ανάθεση τιμών βάσει συνθηκών.Τα αντίστοιχα Testbenches (_tb.vhd) λειτουργούν ως εικονικές γεννήτριες σημάτων, όπου μέσω ενός for loop παράγουν διαδοχικά όλους τους δυνατούς συνδυασμούς εισόδου από το "000" έως το "111" με χρονική καθυστέρηση 20 ns ανά βήμα. Στις κυματομορφές του Vivado, αναμένουμε να δούμε το σήμα s_enc να αυξάνεται σταδιακά και το σήμα s_dec να εμφανίζει έναν "ολισθαίνοντα" άσο (one-hot encoding), όπου σε κάθε χρονικό διάστημα μόνο ένα bit της εξόδου είναι '1', επιβεβαιώνοντας την ορθή λειτουργία της αποκωδικοποίησης.

➢ B.2 – right_left_shifter (version 1 - case)

Κώδικας VHDL – Source Code

```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3.
 4. entity shift_reg is
 5.
 6.     port (
 7.
 8.         clk,rst,si,en,pl,mode:      in std_logic;
 9.         din:                        in std_logic_vector(3 downto 0);
10.         so:                         out std_logic);
11.
12. end shift_reg;
13.
14.
15. architecture rtl of shift_reg is
16.     signal dff  : std_logic_vector(3 downto 0);
17.     signal temp : std_logic := '0';
18. begin
19.     edge: process (clk,rst)
20.     begin
21.
22.         if rst='0' then
23.             dff<=(others=>'0');
24.
25.         elsif clk'event and clk='1' then
26.             if pl='1' then
27.                 dff<=din;
28.
29.             elsif en='1' then
30.                 case mode is
31.                     when '0' =>
32.                         temp <= dff(0);
33.                         dff <= si & dff(3 downto 1);
34.                     when '1' =>
35.                         dff <= dff(2 downto 0) & si;
36.                         temp <= dff(3);
37.                     when others =>
38.                         dff <= "0000";
39.                         temp <= '0';
40.                 end case;
41.             end if;
42.         end if;
43.     end process;
44.     so <= temp;
45.     --so <= dff(0) when mode='0' else dff(3);
46.
47. end rtl;
48.
```

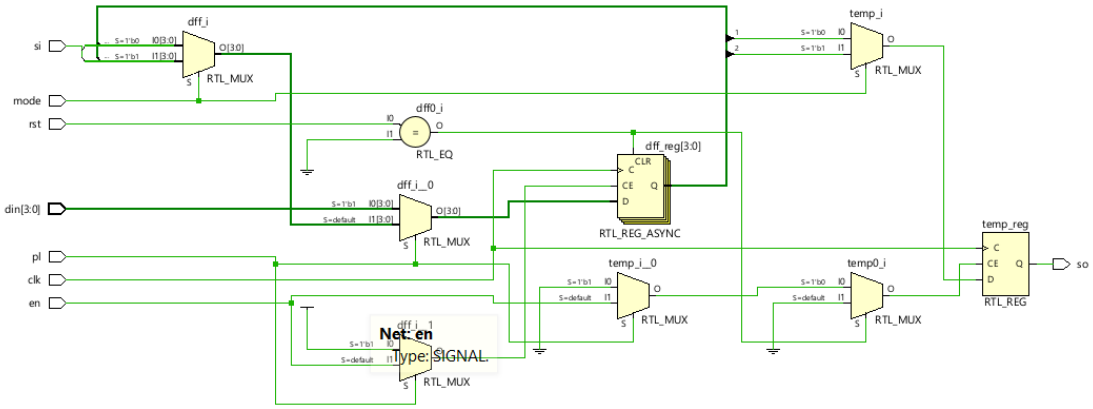
Κώδικας VHDL – Testbench

```
 1. library ieee;
 2. use ieee.std_logic_1164.all;
 3. use ieee.numeric_std.all;
 4.
 5. entity shift_reg_tb is
 6. end entity;
 7.
 8. architecture tb of shift_reg_tb is
```

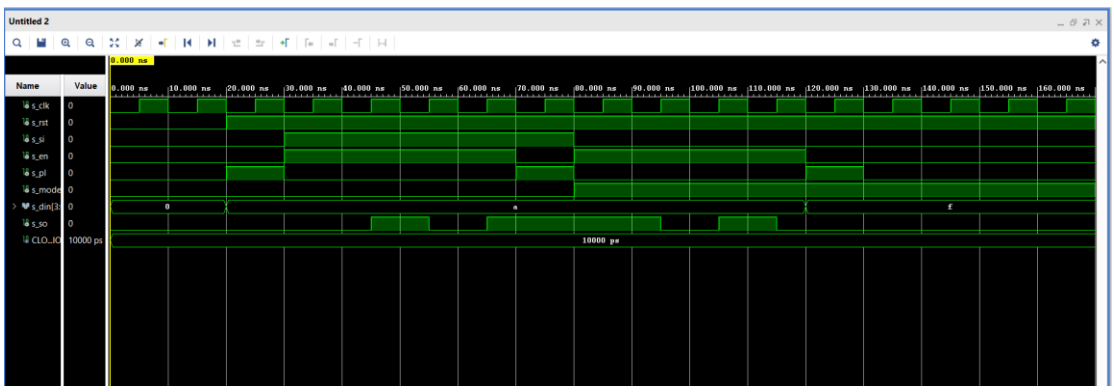
```
    ------------------------------------------------------------
    -- COMPONENT
    ------------------------------------------------------------
    component shift_reg is

        port (

            clk,rst,si,en,pl,mode:      in std_logic;
            din:                        in std_logic_vector(3 downto 0);
            so:                         out std_logic);

    end component;

    ------------------------------------------------------------
    -- SIGNALS
    ------------------------------------------------------------
    signal s_clk,s_rst,s_si,s_en,s_pl,s_mode    : std_logic := '0';
    signal s_din                                : std_logic_vector(3 downto 0) := (others => '0');
    signal s_so                                 : std_logic;

    ------------------------------------------------------------
    -- CONSTANTS
    ------------------------------------------------------------
    constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT : shift_reg
        port map (
            clk     => s_clk,
            rst     => s_rst,
            si      => s_si,
            en      => s_en,
            pl      => s_pl,
            mode    => s_mode,
            din     => s_din,
            so      => s_so
        );


    GEN_CLK : process
    begin
        s_clk <= '0';
        wait for (CLOCK_PERIOD / 2);
        s_clk <= '1';
        wait for (CLOCK_PERIOD / 2);
    end process;

    STIMULUS : process
    begin

        -- INITIALIZE SIGNALS
        s_en <= '0'; --NO SHIFTS YET
        s_din <= (others => '0');
        wait for (1*CLOCK_PERIOD);

        --CHECK RESET
        s_rst <= '0';
        wait for (1*CLOCK_PERIOD);
        s_rst <= '1';

        --CHECK PARALLEL LOAD
        s_din <= "1010";
        s_pl <= '1';
        wait for (1*CLOCK_PERIOD);
        s_pl <= '0';

        --CHECK RIGHT SHIFT (s_en = '1', s_mode = '0')
        s_mode <= '0';
```

```
        s_en   <= '1';
        s_si   <= '1';
        wait for (4*CLOCK_PERIOD);

        --CHECK PARALLEL LOAD
        s_en <= '0';
        s_pl <= '1';
        wait for (1*CLOCK_PERIOD);
        s_pl <= '0';

        --CHECK LEFT SHIFT (s_en = '1', s_mode = '1')
        s_mode <= '1';
        s_en <= '1';
        s_si   <= '0';
        wait for (4*CLOCK_PERIOD);

        --HALT SHIFT EXPERIMENT
        s_en <= '0';
        --PARALLEL LOAD (1111)
        s_din <= "1111";
        s_pl <= '1';
        wait for (1*CLOCK_PERIOD);
        s_pl <= '0';
        wait for (4*CLOCK_PERIOD);

        wait;
    end process;


end architecture;
```

RTL ANALYSIS – Schematic



SIMULATION



➢ B.2 – right_left_shifter (version 2 - if)

Κώδικας VHDL – Source Code

```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3.
 4. entity shift_reg_alt is
 5.
 6.     port (
 7.
 8.         clk,rst,si,en,pl,mode:      in std_logic;
 9.         din:                        in std_logic_vector(3 downto 0);
10.         so:                         out std_logic);
11.
12. end shift_reg_alt;
13.
14.
15. architecture rtl of shift_reg_alt is
16.     signal dff  : std_logic_vector(3 downto 0);
17.     signal temp : std_logic;
18. begin
19.     edge: process (clk,rst)
20.     begin
21.
22.         if rst='0' then
23.             dff<=(others=>'0');
24.
25.         elsif clk'event and clk='1' then
26.             if pl='1' then
27.                 dff<=din;
28.
29.             elsif en='1' then
30.                 if mode = '0' then
31.                     temp <= dff(0);
32.                     dff <= si & dff(3 downto 1);
33.                 elsif mode = '1' then
34.                     temp <= dff(3);
35.                     dff <= dff(2 downto 0) & si;
36.                 else
37.                     temp <= '0';
38.                     dff <= "0000";
39.                 end if;
40.             end if;
41.         end if;
42.     end process;
43.     so <= temp;
44.
45. end rtl;
46.
```

Σύντομη Περιγραφή Υλοποίησης

Τα παρεχόμενα αρχεία υλοποιούν έναν αμφίδρομο καταχωρητή ολίσθησης 4-bit (shift_reg) με δυνατότητα παράλληλης φόρτωσης (pl), σειριακής εισόδου (si) και ελέγχου κατεύθυνσης μέσω του σήματος mode. Η κύρια διαφορά μεταξύ της κανονικής και της alt υλοποίησης εντοπίζεται στη δομή ελέγχου εντός του process, όπου η πρώτη χρησιμοποιεί τη δομή case και η δεύτερη εμφωλευμένες if-elsif

προτάσεις, αν και λειτουργικά παραμένουν ταυτόσημες.Η σχεδιαστική επιλογή του σήματος temp εντός του συγχρονισμένου process είναι καθοριστική: καθώς η ανάθεση temp <= dff(0) (για δεξιά ολίσθηση) ή temp <= dff(3) (για αριστερή) γίνεται την ίδια χρονική στιγμή με την ολίσθηση του dff, η έξοδος so εμφανίζει το bit που μόλις εκτοπίστηκε ("υπερχείλισε") από τον καταχωρητή. Αντίθετα, αν χρησιμοποιούνταν η εξωτερική, ασύγχρονη ανάθεση so <= dff(0) (comment out), η έξοδος θα ανταποκρινόταν άμεσα στα περιεχόμενα του καταχωρητή, δείχνοντας το bit που πρόκειται να εξέλθει στον επόμενο κύκλο ρολογιού. Η λειτουργία του mode καθορίζει την κατεύθυνση: όταν mode = '0', εκτελείται δεξιά ολίσθηση (si & dff(3 downto 1)) , ενώ όταν mode = '1', εκτελείται αριστερή ολίσθηση (dff(2 downto 0) & si).Στο simulation, τα testbenches επιβεβαιώνουν τη σωστή λειτουργία μέσω διαδοχικών φάσεων: αρχικοποίηση με rst, παράλληλη φόρτωση της τιμής "1010" και στη συνέχεια έλεγχο των ολισθήσεων όπου οι κυματομορφές αναμένεται να δείξουν τη σειριακή μετατόπιση των δεδομένων προς την επιλεγμένη κατεύθυνση.Θα ήθελες να αναλύσουμε πώς θα μπορούσε να τροποποιηθεί ο κώδικας ώστε ο καταχωρητής να λειτουργεί ως κυκλικός (ring counter);

- B.3 – up_&_down_counter

Κώδικας VHDL – Source Code

```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3. use IEEE.std_logic_unsigned.all;
 4.
 5. entity count3_1 is
 6.     port(
 7.         clk,resetn,count_en,mode : in std_logic;
 8.         sum : out std_logic_vector(2 downto 0);
 9.         cout : out std_logic);
10.     end;
11.
12. architecture rtl_nolimit of count3_1 is
13.     signal count: std_logic_vector(2 downto 0);
14. begin
15.     process(clk, resetn)
16.     begin
17.         if resetn='0' then
18.         -- Κώδικας για την περίπτωση του reset (ενεργό χαμηλά)
19.             count <= (others=>'0');
```

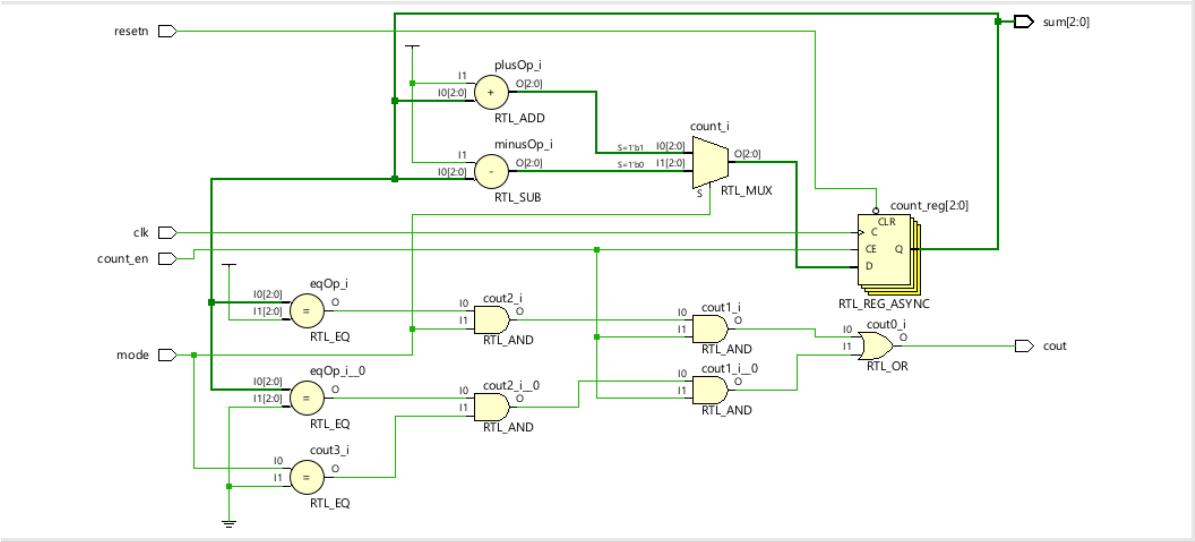
```
        elsif clk'event and clk='1' then
            -- Μέτρηση μόνο αν count_en = 1
            if count_en='1' then
                case mode is
                    when '1' =>
                        count <= count+1;
                    when '0' =>
                        count <= count-1;
                    when others =>
                        count <= "000";
                end case;
            end if;
        end if;
    end process;
    -- Ανάθεση τιμών στα σήματα εξόδου
    sum <= count;
    cout <= '1' when (count = "111" and mode = '1' and count_en = '1') or
                     (count = "000" and mode = '0' and count_en = '1')
                     else '0';

end rtl_nolimit;

```

Κώδικας VHDL – Testbench

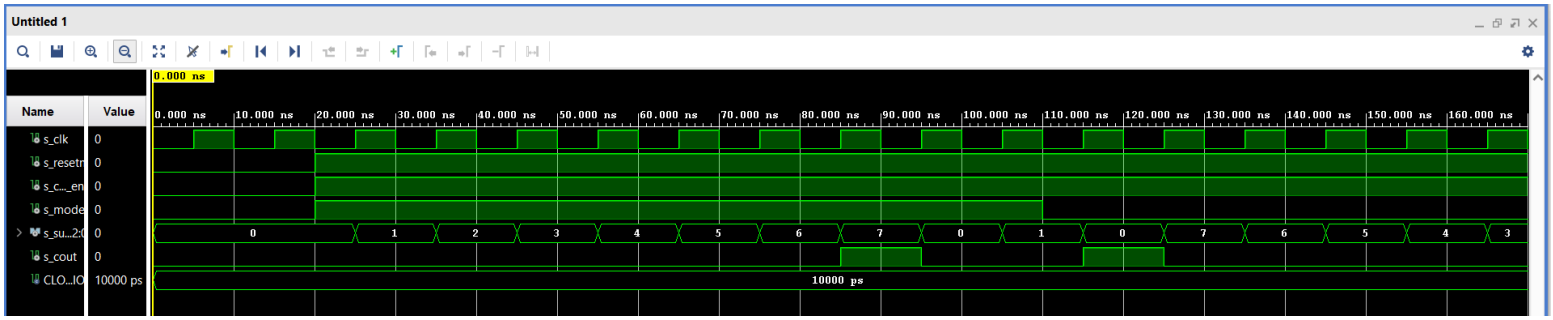
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity rtl1_tb is
end entity;

architecture tb of rtl1_tb is
    ---------------------------------------------------------------
    -- COMPONENT
    ---------------------------------------------------------------
    component count3_1 is

        port (

            clk,resetn,count_en : in std_logic;
            mode                : in std_logic; --COMMENT OUT IF B3_2
            --modulo              : in std_logic_vector(2 downto 0); --COMMENT OUT IF B3_1
            sum                 : out std_logic_vector(2 downto 0);
            cout                : out std_logic
        );

    end component;

    ---------------------------------------------------------------
    -- SIGNALS
    ---------------------------------------------------------------
    signal s_clk,s_resetn,s_count_en            : std_logic := '0';
    signal s_mode                               : std_logic := '0';  --COMMENT OUT IF B3_2
    --signal s_modulo                             : std_logic_vector(2 downto 0) := (others => '0');  --COMMENT OUT IF B3_1
    signal s_sum                                : std_logic_vector(2 downto 0);
    signal s_cout                               : std_logic;

    ---------------------------------------------------------------
    -- CONSTANTS
    ---------------------------------------------------------------
    constant CLOCK_PERIOD : time := 10 ns;

```

```
begin

    DUT : count3_1
        port map (
            clk         => s_clk,
            resetn      => s_resetn,
            count_en    => s_count_en,
            mode        => s_mode,
            --modulo      => s_modulo,  --COMMENT OUT IF B3_2
            sum         => s_sum,
            cout        => s_cout
        );


    GEN_CLK : process
    begin
        s_clk <= '0';
        wait for (CLOCK_PERIOD / 2);
        s_clk <= '1';
        wait for (CLOCK_PERIOD / 2);
    end process;

    STIMULUS : process
    begin

        -- INITIALIZE SIGNALS
        s_count_en <= '0';
        --s_modulo <= (others => '0');
        wait for (1*CLOCK_PERIOD);

        --CHECK RESET
        s_resetn <= '0';
        wait for (1*CLOCK_PERIOD);
        s_resetn <= '1';

        ----------------
        --CHECK B3_1--
        ----------------

        --CHECK COUNT UP
        s_count_en <= '1';
        s_mode <= '1';
        wait for (9*CLOCK_PERIOD);

        --CHECK COUNT DOWN
        s_mode   <= '0';
        wait for (9*CLOCK_PERIOD);

        s_count_en <= '0';

        ----------------
        --CHECK B3_2--
        ----------------
        --s_modulo <= "101";
        --s_count_en <= '1';
        --wait for (10*CLOCK_PERIOD);

        --s_count_en <= '0';
        wait;
    end process;


end architecture;

```

RTL ANALYSIS – Schematic



SIMULATION



➢ B.3 – mod_counter

Κώδικας VHDL – Source Code

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity count3_2 is
    port(
        clk,resetn,count_en : in std_logic;
        modulo              : in std_logic_vector(2 downto 0);
        sum                 : out std_logic_vector(2 downto 0);
        cout                : out std_logic
    );
end;

architecture rtl_limit of count3_2 is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
```

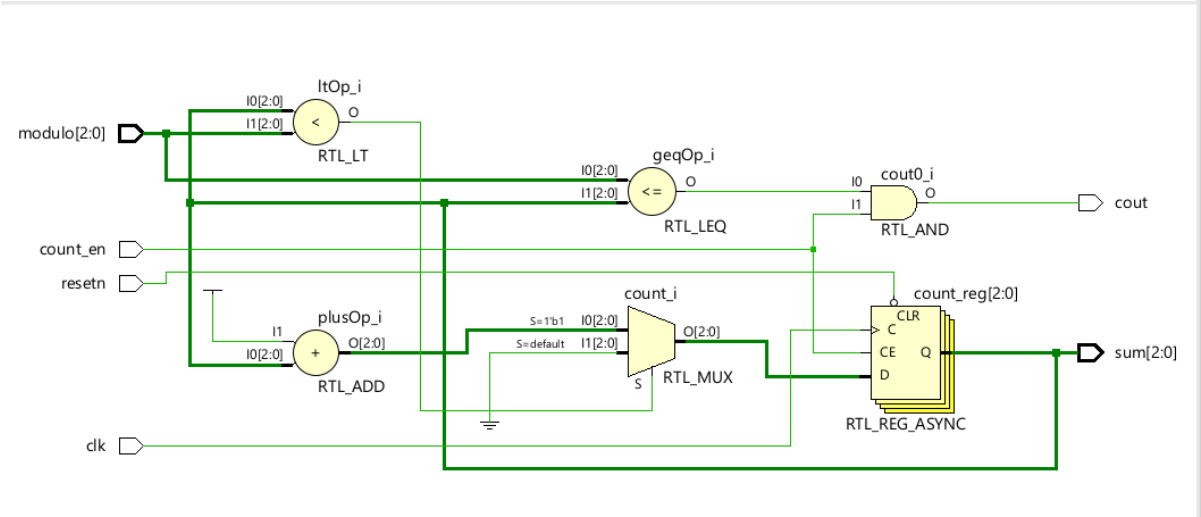
```
        if resetn='0' then
        -- Ασύγχρονος μηδενισμός
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en = '1' then
            -- Μέτρηση μόνο αν count_en=’1’
                if count<modulo then
                -- Αυξάνουμε το μετρητή μόνο αν
                -- δεν είναι modulo
                    count <= count+1;
                else
            -- Αλλιώς τον μηδενίζουμε
                    count<=(others=>'0');
                end if;
            end if;
        end if;
    end process;
    sum<= count;
    cout <= '1' when count>=modulo and count_en='1' else '0';

end rtl_limit;

```

Κώδικας VHDL – Testbench

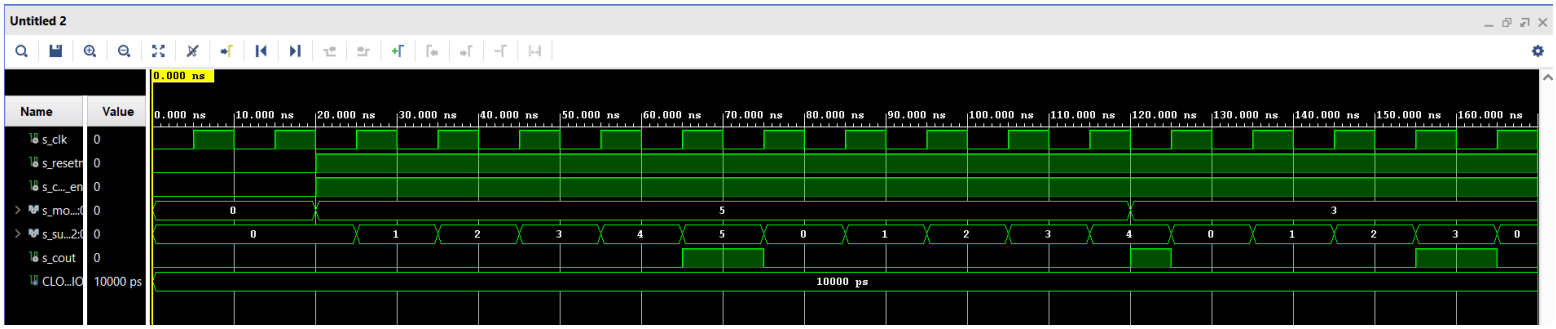
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity rtl2_tb is
end entity;

architecture tb of rtl2_tb is
    ---------------------------------------------------------------
    -- COMPONENT
    ---------------------------------------------------------------
    component count3_2 is

        port (

            clk,resetn,count_en : in std_logic;
            --mode                : in std_logic; --COMMENT OUT IF B3_2
            modulo              : in std_logic_vector(2 downto 0); --COMMENT OUT IF B3_1
            sum                 : out std_logic_vector(2 downto 0);
            cout                : out std_logic
        );

    end component;

    ---------------------------------------------------------------
    -- SIGNALS
    ---------------------------------------------------------------
    signal s_clk,s_resetn,s_count_en            : std_logic := '0';
    --signal s_mode                               : std_logic := '0';  --COMMENT OUT IF B3_2
    signal s_modulo                             : std_logic_vector(2 downto 0) := (others => '0');  --COMMENT OUT IF B3_1
    signal s_sum                                : std_logic_vector(2 downto 0);
    signal s_cout                               : std_logic;

    ---------------------------------------------------------------
    -- CONSTANTS
    ---------------------------------------------------------------
    constant CLOCK_PERIOD : time := 10 ns;

```

```
39. begin
40.
41.     DUT : count3_2
42.         port map (
43.             clk         => s_clk,
44.             resetn      => s_resetn,
45.             count_en    => s_count_en,
46.             --mode        => s_mode,
47.             modulo      => s_modulo,  --COMMENT OUT IF B3_2
48.             sum         => s_sum,
49.             cout        => s_cout
50.         );
51.
52.
53.     GEN_CLK : process
54.     begin
55.         s_clk <= '0';
56.         wait for (CLOCK_PERIOD / 2);
57.         s_clk <= '1';
58.         wait for (CLOCK_PERIOD / 2);
59.     end process;
60.
61.     STIMULUS : process
62.     begin
63.
64.         -- INITIALIZE SIGNALS
65.         s_count_en <= '0';
66.         s_modulo <= (others => '0');
67.         wait for (1*CLOCK_PERIOD);
68.
69.         --CHECK RESET
70.         s_resetn <= '0';
71.         wait for (1*CLOCK_PERIOD);
72.         s_resetn <= '1';
73.
74.         ----------------
75.         --CHECK B3_1--
76.         ----------------
77.
78.         --CHECK COUNT UP
79.         --s_count_en <= '1';
80.         --s_mode <= '1';
81.         --wait for (8*CLOCK_PERIOD);
82.
83.         --CHECK COUNT DOWN
84.         --s_mode   <= '0';
85.         --wait for (8*CLOCK_PERIOD);
86.
87.         -- <= '0';
88.
89.         ----------------
90.         --CHECK B3_2--
91.         ----------------
92.         s_modulo <= "101";
93.         s_count_en <= '1';
94.         wait for (10*CLOCK_PERIOD);
95.
96.         s_modulo <= "011";
97.         wait for (6*CLOCK_PERIOD);
98.
99.         s_count_en <= '0';
100.        wait;
101.    end process;
102.
103.
104. end architecture;
105.
```

RTL ANALYSIS – Schematic



SIMULATION



Σύντομη Περιγραφή Υλοποίησης

Στα αρχεία count3_1 και count3_2 υλοποιούνται δύο τύποι μετρητών 3-bit με διαφορετικούς μηχανισμούς ελέγχου. Στον πρώτο μετρητή , το σήμα mode καθορίζει την κατεύθυνση, λειτουργώντας ως up counter όταν είναι '1' και ως down counter όταν είναι '0'. Στον δεύτερο μετρητή , το σήμα modulo εισάγει ένα δυναμικό ανώτατο όριο, επιτρέποντας στο κύκλωμα να μηδενίζει αυτόματα μόλις η μέτρηση φτάσει την τιμή αυτή.Κομβική σχεδιαστική επιλογή στο count3_2 είναι ο έλεγχος count >= modulo για την παραγωγή του κρατουμένου (cout) και τον μηδενισμό. Η χρήση του τελεστή >= αντί του = εξασφαλίζει ότι αν το modulo μειωθεί δυναμικά σε τιμή μικρότερη από την τρέχουσα μέτρηση, ο μετρητής θα την αναγνωρίσει αμέσως, θα ενεργοποιήσει το cout και θα επανέλθει στο

"000" στον επόμενο κύκλο. Με αυτόν τον τρόπο αποφεύγεται ο κίνδυνος να "προσπεράσει" ο μετρητής το όριο και να συνεχίσει ανεξέλεγκτα τη μέτρηση μέχρι την επόμενη φυσική υπερχείλιση.Στο simulation του Vivado, βάσει των test benches, αναμένουμε να δούμε:Για τον count3_1: Το σήμα s_sum να αυξάνεται διαδοχικά ανά CLOCK_PERIOD όταν το s_mode είναι '1' και να μειώνεται όταν το s_mode γίνεται '0', με το s_cout να εμφανίζει παλμό στις ακραίες τιμές ("111" ή "000" αντίστοιχα).Για τον count3_2: Το σήμα s_sum να μετράει ανοδικά μέχρι την τιμή που ορίζει το s_modulo. Μόλις η μέτρηση φτάσει ή ξεπεράσει το όριο, το s_cout θα ενεργοποιηθεί και στον επόμενο κύκλο το s_sum θα μηδενιστεί, επιβεβαιώνοντας τη λειτουργία του δυναμικού ορίου.

Μέρος Β

- Ερώτημα 1 – half adder

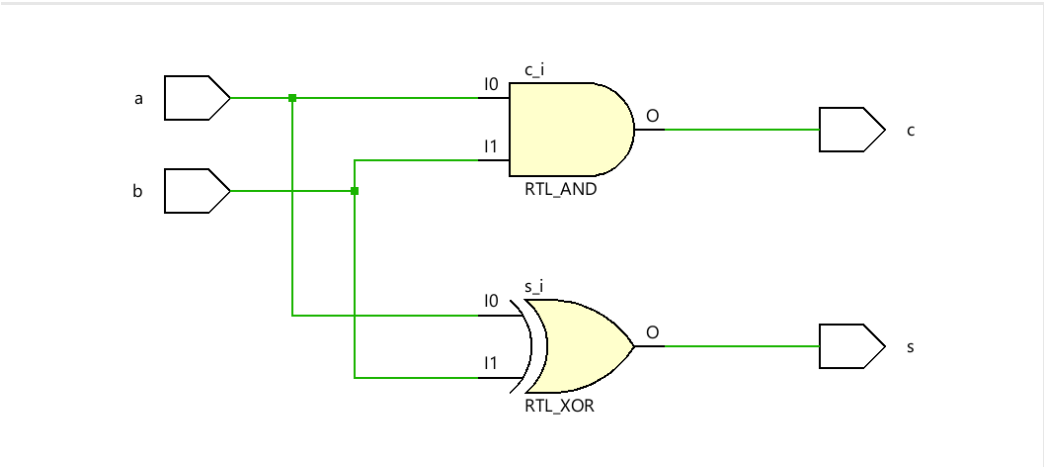
Κώδικας VHDL – Source Code

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity half_adder is
    port (
        a,b : in std_logic;
        s,c   : out std_logic
    );
end half_adder;

architecture dataflow_arch of half_adder is
begin
    s <= a xor b;
    c <= a and b;

end dataflow_arch;

```

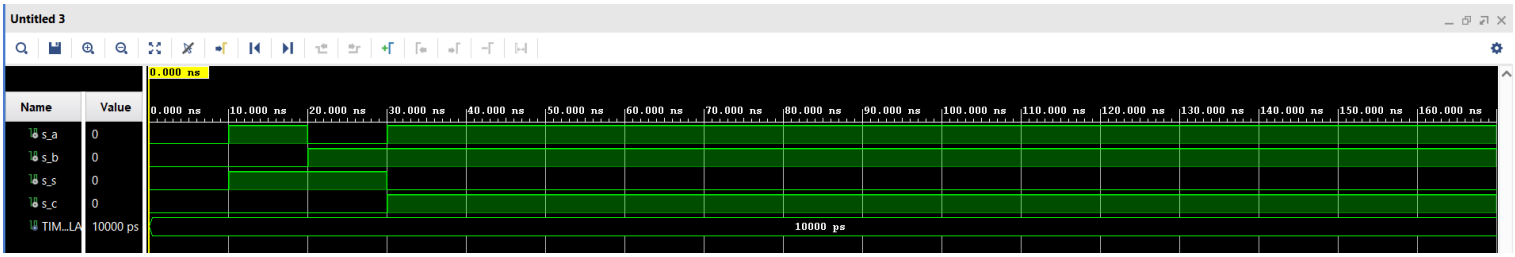
Κώδικας VHDL – Testbench

```
 1. library ieee;
 2. use ieee.std_logic_1164.all;
 3. use ieee.numeric_std.all;
 4.
 5. entity half_adder_tb is
 6. end entity;
 7.
 8. architecture tb of half_adder_tb is
 9.
10.     component half_adder
11.         port (
12.             a,b : in std_logic;
13.             s,c : out std_logic
14.         );
15.     end component;
16.
17.     signal s_a, s_b : std_logic := '0';
18.     signal s_s, s_c : std_logic;
19.     --signal input_vector : std_logic_vector(1 downto 0);
20.
21.     constant TIME_DELAY : time := 10 ns;
22.
23. begin
24.
25.     DUT : half_adder
26.         port map (
27.             a    => s_a,
28.             b    => s_b,
29.             s    => s_s,
30.             c    => s_c
31.         );
32.
33.
34.     STIMULUS : process
35.         variable input_vector : std_logic_vector(1 downto 0);
36.     begin
37.         for i in 0 to 3 loop
38.             input_vector := std_logic_vector(to_unsigned(i, 2));
39.             --wait for 1 ns; --necessary stall to pass the values to input vector
40.
41.             s_a <= input_vector(0);
42.             s_b <= input_vector(1);
43.             wait for (1*TIME_DELAY);
44.         end loop;
45.
46.         wait;
47.     end process;
48. end tb;
49.
```

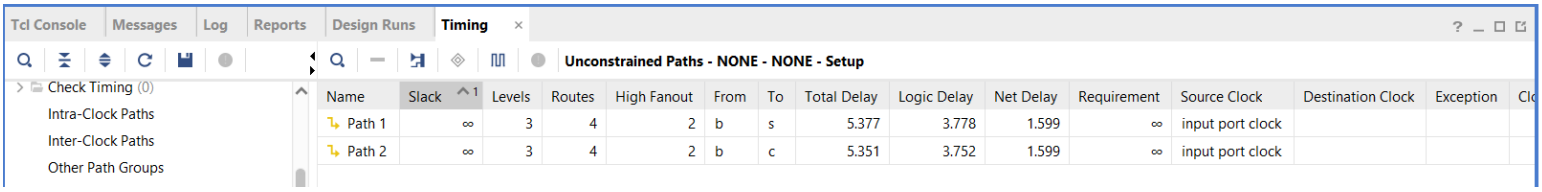
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	b	s	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	b	c	5.351	3.752	1.599	∞	input port clock		

Σύντομη Περιγραφή Υλοποίησης

Η υλοποίηση του Half Adder αποτελεί το θεμέλιο λίθο της ιεραρχικής σχεδίασης, καθώς περιγράφει τη βασική πρόσθεση δύο bit εισόδου (a, b) παράγοντας το άθροισμα s (μέσω της πύλης XOR) και το κρατούμενο εξόδου c (μέσω της πύλης AND). Ο πηγαίος κώδικας ακολουθεί τη μέθοδο Dataflow (ροής δεδομένων), η οποία είναι η πλέον κατάλληλη για αυτό το επίπεδο, καθώς οι Boolean εξισώσεις μεταφέρονται άμεσα σε εντολές ταυτόχρονης ανάθεσης, προσφέροντας μια καθαρή αντιστοίχιση μεταξύ λογικής και υλικού. Η RTL σχεδίαση που προκύπτει από τη σύνθεση αποτελείται αποκλειστικά από τις δύο αυτές λογικές πύλες συνδεδεμένες παράλληλα στις εισόδους, εξασφαλίζοντας ελάχιστη καθυστέρηση διάδοσης.

Στο simulation του Vivado, βάσει του testbench, αναμένουμε να δούμε τέσσερις διακριτές καταστάσεις στις κυματομορφές ανά TIME_DELAY: όταν οι είσοδοι είναι "00" ή "11" το άθροισμα s παραμένει '0', ενώ στην περίπτωση "11" ενεργοποιείται το κρατούμενο c. Αντίθετα, όταν μόνο μία από τις εισόδους είναι '1' ("01" ή "10"), το σήμα s ανεβαίνει σε υψηλή στάθμη ενώ το c παραμένει στο '0', επιβεβαιώνοντας τον πίνακα αληθείας του κυκλώματος.

➢ Ερώτημα 2 – full adder

Κώδικας VHDL – Source Code

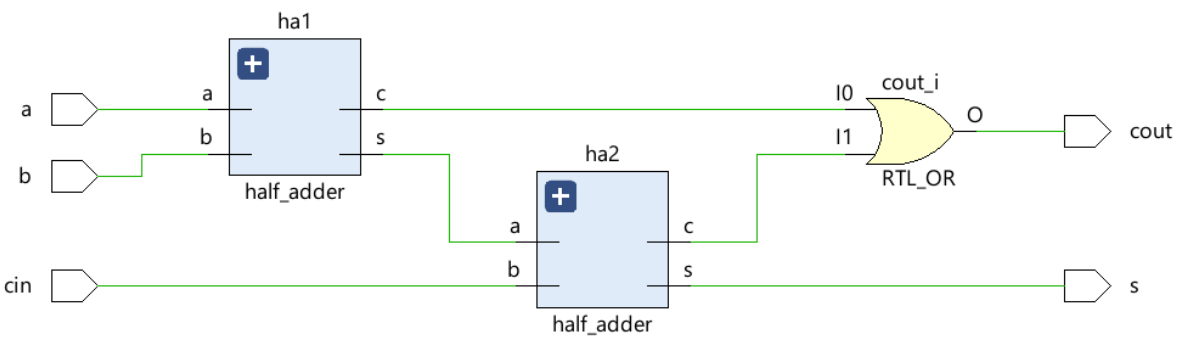
```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3. use IEEE.std_logic_unsigned.all;
 4.
 5. entity full_adder is
 6.     port (
 7.         a,b,cin : in std_logic;
 8.         s,cout  : out std_logic
 9.     );
10. end full_adder;
11.
12. architecture structural_arch of full_adder is
13.
14.     component half_adder is
15.         port (
16.             a,b     : in std_logic;
17.             s,c     : out std_logic
18.         );
19.     end component;
20.
21.     signal s1,c1,c2 : std_logic;
22.
23. begin
24.
25.     ha1 : half_adder
26.         port map (
27.             a => a,
28.             b => b,
29.             s => s1,
30.             c => c1
31.         );
32.
33.     ha2 : half_adder
34.         port map (
35.             a => s1,
36.             b => cin,
37.             s => s,
38.             c => c2
39.         );
40.
41.     cout <= c1 or c2;
42.
43.
44. end structural_arch;
45.
```

Κώδικας VHDL – Testbench

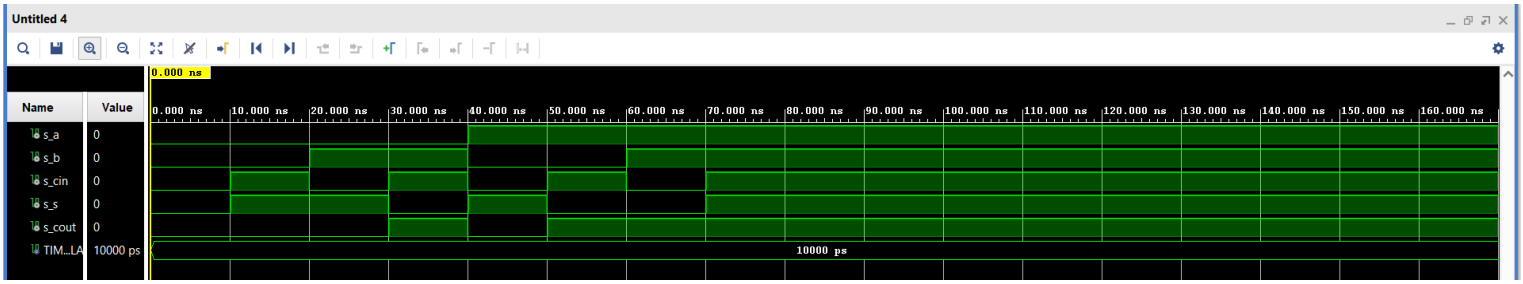
```
 1. library ieee;
 2. use ieee.std_logic_1164.all;
 3. use ieee.numeric_std.all;
 4.
 5. entity full_adder_tb is
 6. end entity;
 7.
 8. architecture tb of full_adder_tb is
 9.
10.     component full_adder
11.         port (
```

```
            a,b,cin : in std_logic;
            s,cout  : out std_logic
        );
    end component;

    signal s_a, s_b, s_cin  : std_logic := '0';
    signal s_s, s_cout      : std_logic;
    --signal input_vector     : std_logic_vector(2 downto 0);

    constant TIME_DELAY : time := 10 ns;

begin

    DUT : full_adder
        port map (
            a       => s_a,
            b       => s_b,
            cin     => s_cin,
            s       => s_s,
            cout    => s_cout
        );


    STIMULUS : process
        variable input_vector : std_logic_vector(2 downto 0);
    begin
        for i in 0 to 7 loop
            input_vector := std_logic_vector(to_unsigned(i, 3));
            --wait for 5 ns;

            s_a     <= input_vector(2);
            s_b     <= input_vector(1);
            s_cin   <= input_vector(0);
            wait for (1*TIME_DELAY);

        end loop;

        wait;
    end process;
end tb;

```

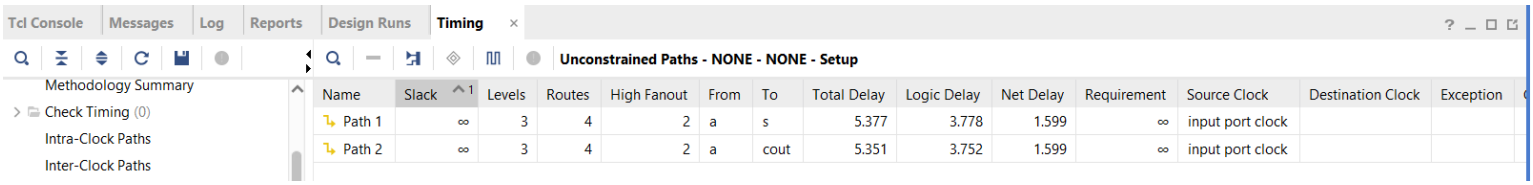
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	a	s	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	a	cout	5.351	3.752	1.599	∞	input port clock		

Σύντομη Περιγραφή Υλοποίησης

Η υλοποίηση του Full Adder (Πλήρης Αθροιστής) αποτελεί το επόμενο βήμα στην ιεραρχική σχεδίαση, καθώς επιτρέπει την πρόσθεση τριών bit (a, b, cin) ενσωματώνοντας το κρατούμενο από προηγούμενη βαθμίδα. Ο κώδικας ακολουθεί Structural (Δομική) λογική, όπου η οντότητα συντίθεται από τη διασύνδεση δύο Half Adders και μιας πύλης OR: ο πρώτος HA προσθέτει τα a και b, ενώ ο δεύτερος προσθέτει το μερικό άθροισμα με το cin. Η RTL σχεδίαση απεικονίζει αυτή ακριβώς την ιεραρχία, εμφανίζοντας δύο διακριτά blocks (τα components του HA) και την πύλη OR που συλλέγει τα επιμέρους κρατούμενα. Στις κυματομορφές του Vivado, αναμένουμε να δούμε την πλήρη επιβεβαίωση του πίνακα αληθείας 8 καταστάσεων, όπου για παράδειγμα στην είσοδο "111" τόσο το άθροισμα s όσο και το κρατούμενο cout θα βρίσκονται σε υψηλή στάθμη.

Όσον αφορά την ανάλυση χρονισμού, το Critical Path (κρίσιμο μονοπάτι) εντοπίζεται στη διαδρομή από τις εισόδους a ή b προς την έξοδο cout, καθώς το σήμα πρέπει να διέλθει διαδοχικά από την πύλη AND του πρώτου HA και στη συνέχεια από την πύλη OR του Full Adder. Στο Timing Report, η καθυστέρηση αυτή (propagation delay) είναι η

σημαντικότερη, καθώς σε μια αλυσίδα αθροιστών, το κρατούμενο πρέπει να "ταξιδέψει" μέσα από κάθε βαθμίδα, καθορίζοντας έτσι τη μέγιστη συχνότητα λειτουργίας ολόκληρου του ψηφιακού συστήματος.

- Ερώτημα 3 – 4bit Parallel Adder

Κώδικας VHDL – Source Code

```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3. use IEEE.std_logic_unsigned.all;
 4.
 5. entity pa4 is
 6.     port (
 7.         a       : in std_logic_vector(3 downto 0);
 8.         b       : in std_logic_vector(3 downto 0);
 9.         cin     : in std_logic;
10.         sum     : out std_logic_vector(3 downto 0);
11.         cout    : out std_logic
12.     );
13. end pa4;
14.
15. architecture structural_arch of pa4 is
16.
17.     component full_adder is
18.         port (
19.             a,b,cin     : in std_logic;
20.             s,cout      : out std_logic
21.         );
22.     end component;
23.
24.     signal c_vector : std_logic_vector(2 downto 0);
25.
26. begin
27.
28.     fa1 : full_adder
29.         port map (
30.             a       => a(0),
31.             b       => b(0),
32.             cin     => cin,
33.             s       => sum(0),
34.             cout    => c_vector(0)
35.         );
36.
37.     fa2 : full_adder
38.         port map (
39.             a       => a(1),
40.             b       => b(1),
41.             cin     => c_vector(0),
42.             s       => sum(1),
43.             cout    => c_vector(1)
44.         );
45.
46.
47.     fa3 : full_adder
48.         port map (
49.             a       => a(2),
50.             b       => b(2),
51.             cin     => c_vector(1),
```

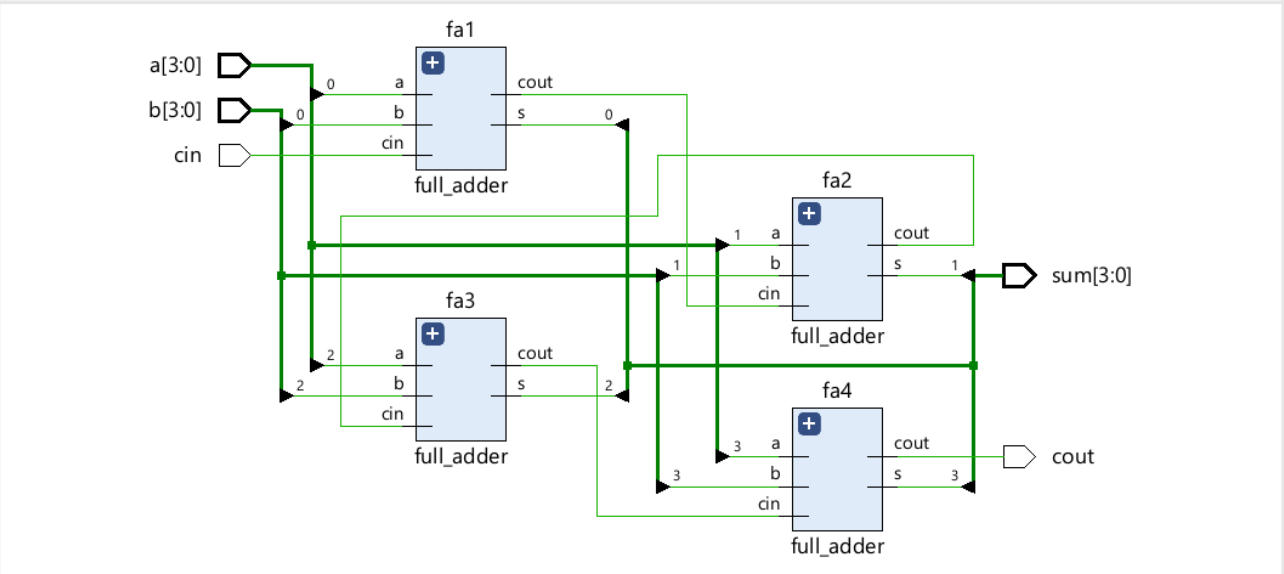
```
            s       => sum(2),
            cout    => c_vector(2)
        );

    fa4 : full_adder
        port map (
            a       => a(3),
            b       => b(3),
            cin     => c_vector(2),
            s       => sum(3),
            cout    => cout
        );


end structural_arch;

```

Κώδικας VHDL – Testbench

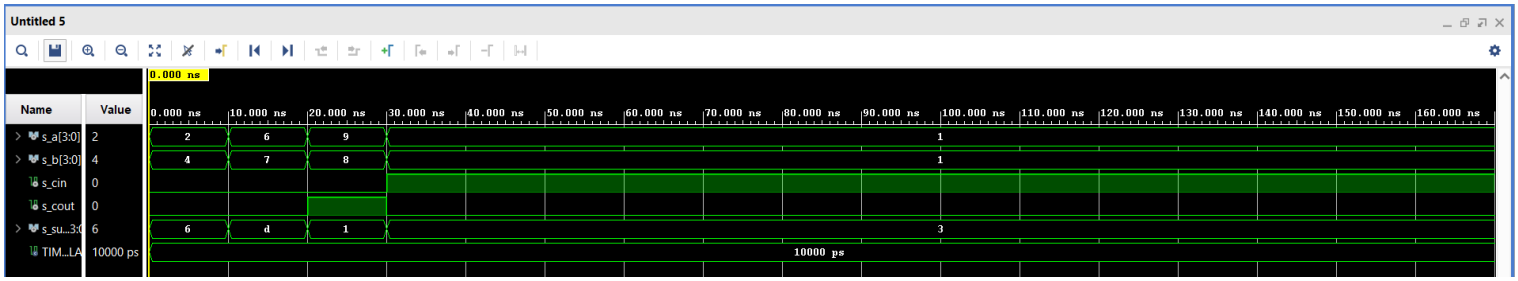
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pa4_tb is
end entity;

architecture tb of pa4_tb is

    component pa4
        port (
            a,b     : in std_logic_vector(3 downto 0);
            cin     : in std_logic;
            sum     : out std_logic_vector(3 downto 0);
            cout    : out std_logic
        );
    end component;

    signal s_a, s_b         : std_logic_vector(3 downto 0) := (others => '0');
    signal s_cin            : std_logic := '0';
    signal s_cout           : std_logic;
    signal s_sum            : std_logic_vector(3 downto 0);

    constant TIME_DELAY : time := 10 ns;

begin

    DUT : pa4
        port map (
            a       => s_a,
            b       => s_b,
            cin     => s_cin,
            sum     => s_sum,
            cout    => s_cout
        );


    STIMULUS : process
    begin
        s_a <= std_logic_vector(to_unsigned(2,4));
        s_b <= std_logic_vector(to_unsigned(4,4));
        wait for (1*TIME_DELAY);

        s_a <= std_logic_vector(to_unsigned(6,4));
        s_b <= std_logic_vector(to_unsigned(7,4));
        wait for (1*TIME_DELAY);

```

```
        s_a <= std_logic_vector(to_unsigned(9,4));
        s_b <= std_logic_vector(to_unsigned(8,4));
        wait for (1*TIME_DELAY);

        s_a     <= std_logic_vector(to_unsigned(1,4));
        s_b     <= std_logic_vector(to_unsigned(1,4));
        s_cin   <= '1';
        wait for (1*TIME_DELAY);

        --s_cin <= std_logic_vector(to_unsigned(0,4));
    --    for i in 0 to 15 loop
    --        for j in 0 to 15 loop
    --            for k in 0 to 1 loop
    --                s_a <= std_logic_vector(to_unsigned(i, 4));
    --                s_b <= std_logic_vector(to_unsigned(j, 4));
    --                if k = 1 then
    --                    s_cin <= '1';
    --                else
    --                    s_cin <= '0';
    --                end if;
    --                wait for (1*TIME_DELAY);
    --            end loop;
    --        end loop;
    --    end loop;

        wait;
    end process;
end tb;

```

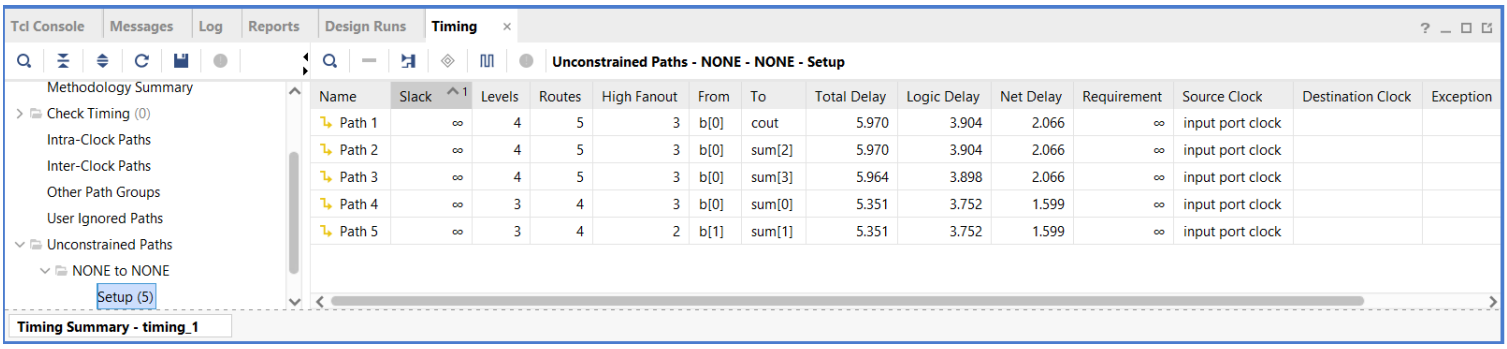
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	4	5	3	b[0]	cout	5.970	3.904	2.066	∞	input port clock		
Path 2	∞	4	5	3	b[0]	sum[2]	5.970	3.904	2.066	∞	input port clock		
Path 3	∞	4	5	3	b[0]	sum[3]	5.964	3.898	2.066	∞	input port clock		
Path 4	∞	3	4	3	b[0]	sum[0]	5.351	3.752	1.599	∞	input port clock		
Path 5	∞	3	4	2	b[1]	sum[1]	5.351	3.752	1.599	∞	input port clock		

Σύντομη Περιγραφή Υλοποίησης

Η υλοποίηση του Parallel Adder 4-bit (pa4) αποτελεί την πρώτη επέκταση σε επίπεδο λέξης (word level), συνδυάζοντας τέσσερις Full Adders σε σειρά μέσω Structural σχεδίασης. Η λογική του κώδικα βασίζεται στη μεταφορά του κρατουμένου από τη μία βαθμίδα στην επόμενη (Ripple Carry), όπου το cout κάθε αθροιστή συνδέεται στο cin του επόμενου μέσω του εσωτερικού σήματος c_vector. Η RTL σχεδίαση αναδεικνύει μια επαναλαμβανόμενη δομή τεσσάρων πανομοιότυπων blocks, αντικατοπτρίζοντας την κλιμακωτή φύση του κυκλώματος. Στις κυματομορφές του Vivado, αναμένουμε να δούμε το συνολικό άθροισμα sum και το τελικό κρατούμενο cout να μεταβάλλονται σύμφωνα με τις εισόδους 4-bit, όπως για παράδειγμα στην πρόσθεση "0110" (6) και "0111" (7) που θα δώσει αποτέλεσμα "1101" (13) με cout μηδέν.

Το Critical Path σε αυτό το κύκλωμα είναι η διαδρομή του κρατουμένου από την πρώτη βαθμίδα (LSB) έως την τελευταία έξοδο cout (Carry Propagation Delay). Καθώς κάθε Full Adder πρέπει να

περιμένει το κρατούμενο από τον προηγούμενο για να οριστικοποιήσει το δικό του αποτέλεσμα, η συνολική καθυστέρηση αυξάνεται γραμμικά με τον αριθμό των bits. Στο Timing Report, αυτή η συσσωρευμένη καθυστέρηση (Worst Case Delay) είναι το πιο κρίσιμο μέγεθος, καθώς αν το ρολόι του συστήματος είναι ταχύτερο από το χρόνο που χρειάζεται το κρατούμενο για να "διατρέξει" όλη την αλυσίδα, το αποτέλεσμα στην έξοδο θα είναι λανθασμένο.

➢ Ερώτημα 4 – BCD Full Adder

Κώδικας VHDL – Source Code

```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3. use IEEE.std_logic_unsigned.all;
 4.
 5. entity bcd_fa is
 6.     port (
 7.         a       : in std_logic_vector(3 downto 0);
 8.         b       : in std_logic_vector(3 downto 0);
 9.         cin     : in std_logic;
10.         sum     : out std_logic_vector(3 downto 0);
11.         cout    : out std_logic
12.     );
13. end bcd_fa;
14.
15. architecture structural_arch of bcd_fa is
16.
17.     component pa4 is
18.         port (
19.             a,b         : in std_logic_vector(3 downto 0);
20.             cin         : in std_logic;
21.             sum         : out std_logic_vector(3 downto 0);
22.             cout        : out std_logic
23.         );
24.     end component;
25.
26.     signal sum1                     : std_logic_vector(3 downto 0);
27.     signal bias                     : std_logic_vector(3 downto 0);
28.     signal c1,useless_c,s_cout      : std_logic;
29.
30. begin
31.
32.     pa1 : pa4
33.         port map (
34.             a       => a,
35.             b       => b,
36.             cin     => cin,
37.             sum     => sum1,
38.             cout    => c1
39.         );
40.
41.     s_cout <= c1 or (sum1(3) and (sum1(2) or sum1(1)));
42.     cout <= s_cout;
43.     bias <= '0' & s_cout & s_cout & '0';
44.     --bias <= "0110" when s_cout = '1'
45.     --        else "0000";
```

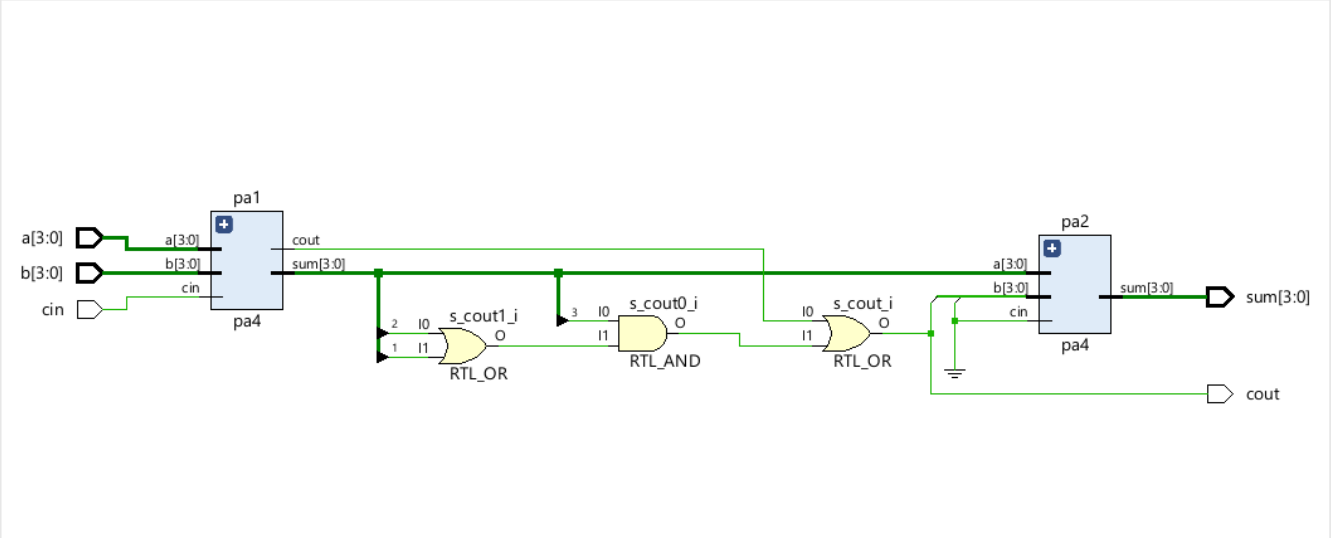
```
46.
47.     pa2 : pa4
48.         port map (
49.             a       => sum1,
50.             b       => bias,
51.             cin     => '0',
52.             sum     => sum,
53.             cout    => useless_c
54.         );
55.
56.
57.
58. end structural_arch;
59.
```

Κώδικας VHDL – Testbench

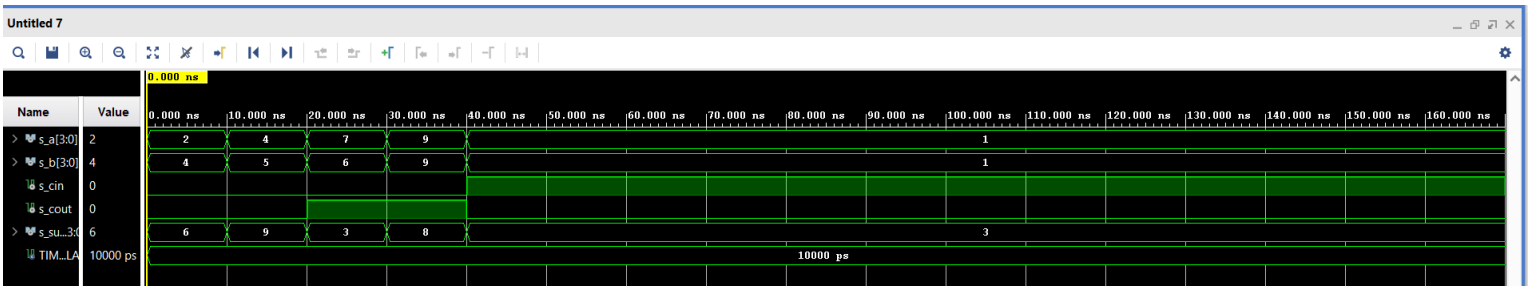
```
 1. library ieee;
 2. use ieee.std_logic_1164.all;
 3. use ieee.numeric_std.all;
 4.
 5. entity bcd_fa_tb is
 6. end entity;
 7.
 8. architecture tb of bcd_fa_tb is
 9.
10.     component bcd_fa
11.         port (
12.             a,b     : in std_logic_vector(3 downto 0);
13.             cin     : in std_logic;
14.             sum     : out std_logic_vector(3 downto 0);
15.             cout    : out std_logic
16.         );
17.     end component;
18.
19.     signal s_a, s_b         : std_logic_vector(3 downto 0) := (others => '0');
20.     signal s_cin            : std_logic := '0';
21.     signal s_cout           : std_logic;
22.     signal s_sum            : std_logic_vector(3 downto 0);
23.
24.     constant TIME_DELAY : time := 10 ns;
25.
26. begin
27.
28.     DUT : bcd_fa
29.         port map (
30.             a       => s_a,
31.             b       => s_b,
32.             cin     => s_cin,
33.             sum     => s_sum,
34.             cout    => s_cout
35.         );
36.
37.
38.     STIMULUS : process
39.
40.     begin
41.         s_a <= std_logic_vector(to_unsigned(2,4));
42.         s_b <= std_logic_vector(to_unsigned(4,4));
43.         wait for (1*TIME_DELAY);
44.
45.         s_a <= std_logic_vector(to_unsigned(4,4));
46.         s_b <= std_logic_vector(to_unsigned(5,4));
47.         wait for (1*TIME_DELAY);
48.
49.         s_a <= std_logic_vector(to_unsigned(7,4));
```

```
        s_b <= std_logic_vector(to_unsigned(6,4));
        wait for (1*TIME_DELAY);

        s_a     <= std_logic_vector(to_unsigned(9,4));
        s_b     <= std_logic_vector(to_unsigned(9,4));
        wait for (1*TIME_DELAY);

        s_a     <= std_logic_vector(to_unsigned(1,4));
        s_b     <= std_logic_vector(to_unsigned(1,4));
        s_cin   <= '1';
        wait for (1*TIME_DELAY);
    --    for i in 0 to 9 loop
    --        for j in 0 to 9 loop
    --            for k in 0 to 1 loop
    --                s_a <= std_logic_vector(to_unsigned(i, 4));
    --                s_b <= std_logic_vector(to_unsigned(j, 4));
    --                if k = 1 then
    --                    s_cin <= '1';
    --                else
    --                    s_cin <= '0';
    --                end if;
    --                wait for (1*TIME_DELAY);
    --            end loop;
    --        end loop;
    --    end loop;

        wait;
    end process;

```

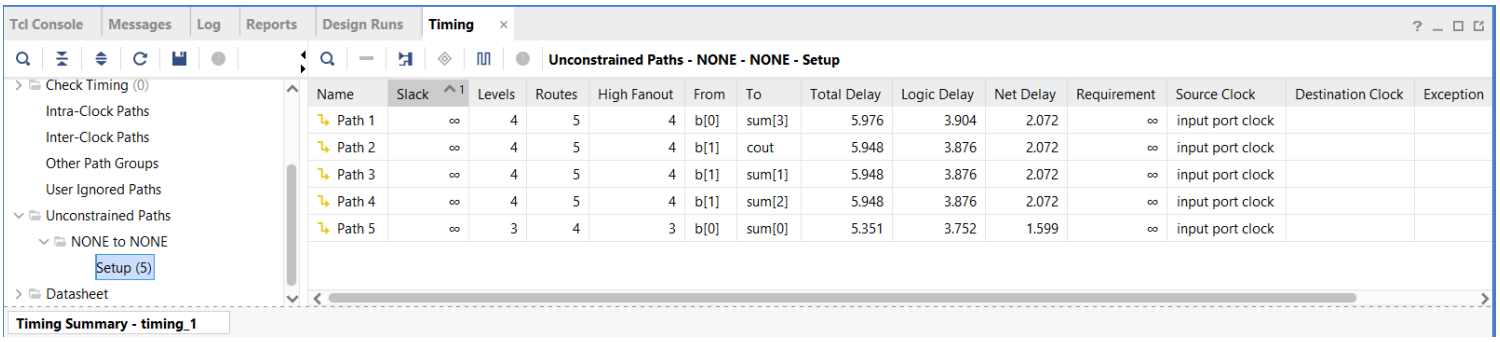
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	4	5	4	b[0]	sum[3]	5.976	3.904	2.072	∞	input port clock		
Path 2	∞	4	5	4	b[1]	cout	5.948	3.876	2.072	∞	input port clock		
Path 3	∞	4	5	4	b[1]	sum[1]	5.948	3.876	2.072	∞	input port clock		
Path 4	∞	4	5	4	b[1]	sum[2]	5.948	3.876	2.072	∞	input port clock		
Path 5	∞	3	4	3	b[0]	sum[0]	5.351	3.752	1.599	∞	input port clock		

Σύντομη Περιγραφή Υλοποίησης

Η υλοποίηση του BCD Adder (Δεκαδικός Αθροιστής) εισάγει τη λογική διόρθωσης στο αποτέλεσμα μιας δυαδικής πρόσθεσης 4-bit, ώστε να παραμένει εντός των ορίων του κώδικα BCD (0-9). Ο κώδικας ακολουθεί Structural σχεδίαση, χρησιμοποιώντας έναν Parallel Adder (pa4) για την αρχική πρόσθεση των a και b και έναν δεύτερο pa4 που προσθέτει την τιμή διόρθωσης "0110" (6), μόνο όταν το αρχικό άθροισμα υπερβαίνει το 9 ή παράγεται κρατούμενο. Η RTL σχεδίαση αποκαλύπτει μια πιο σύνθετη δομή όπου το κρατούμενο εξόδου (cout) παράγεται από μια συνδυαστική λογική που ανιχνεύει τιμές > 9, η οποία με τη σειρά της ενεργοποιεί τη διορθωτική πρόσθεση στη δεύτερη βαθμίδα. Στις κυματομορφές του Vivado, περιμένουμε να δούμε ότι για εισόδους όπως "1000" (8) και "1000" (8), το τελικό άθροισμα sum θα είναι "0110" (6) με cout '1', αναπαριστώντας σωστά τον αριθμό 16 σε μορφή BCD, αντί για το δυαδικό "10000".

Το Critical Path σε αυτό το κύκλωμα επιμηκύνεται σημαντικά, καθώς περιλαμβάνει τη διαδρομή από τις εισόδους, μέσω του πρώτου αθροιστή, στη συνέχεια μέσω της λογικής ανίχνευσης (πύλες AND/OR) και τέλος μέσω του δεύτερου αθροιστή για τη διόρθωση. Στο Timing Report, η καθυστέρηση αυτή είναι σχεδόν διπλάσια από έναν απλό αθροιστή, καθιστώντας τον BCD adder μια πιο αργή μονάδα. Η ανάλυση χρονισμού αναδεικνύει ότι η απόφαση για το αν θα γίνει διόρθωση είναι το πιο "στενό" σημείο (bottleneck) που καθορίζει τη συνολική ταχύτητα απόκρισης του κυκλώματος.

➢ Ερώτημα 5 – BCD Full Adder

Κώδικας VHDL – Source Code

```
1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3.
 4. entity bcd_pa4 is
 5.     port (
 6.         a    : in  std_logic_vector(15 downto 0);
 7.         b    : in  std_logic_vector(15 downto 0);
 8.         cin  : in  std_logic;
 9.         sum  : out std_logic_vector(15 downto 0);
10.         cout : out std_logic
11.     );
12. end bcd_pa4;
13.
14. architecture structural of bcd_pa4 is
15.
16.     component bcd_fa is
17.         port (
18.             a, b : in  std_logic_vector(3 downto 0);
19.             cin  : in  std_logic;
20.             sum  : out std_logic_vector(3 downto 0);
21.             cout : out std_logic
22.         );
23.     end component;
24.
25.
26.     signal c_vector : std_logic_vector(3 downto 1);
27.
28. begin
29.
30.     bcd_pa0: bcd_fa port map (
31.         a    => a(3 downto 0),
32.         b    => b(3 downto 0),
33.         cin  => cin,
34.         sum  => sum(3 downto 0),
35.         cout => c_vector(1)
36.     );
37.
38.     bcd_pa1: bcd_fa port map (
39.         a    => a(7 downto 4),
40.         b    => b(7 downto 4),
41.         cin  => c_vector(1),
42.         sum  => sum(7 downto 4),
43.         cout => c_vector(2)
44.     );
45.
46.     bcd_pa2: bcd_fa port map (
47.         a    => a(11 downto 8),
48.         b    => b(11 downto 8),
49.         cin  => c_vector(2),
50.         sum  => sum(11 downto 8),
51.         cout => c_vector(3)
52.     );
53.
54.     bcd_pa3: bcd_fa port map (
55.         a    => a(15 downto 12),
56.         b    => b(15 downto 12),
57.         cin  => c_vector(3),
58.         sum  => sum(15 downto 12),
59.         cout => cout
60.     );
61.
62.
```

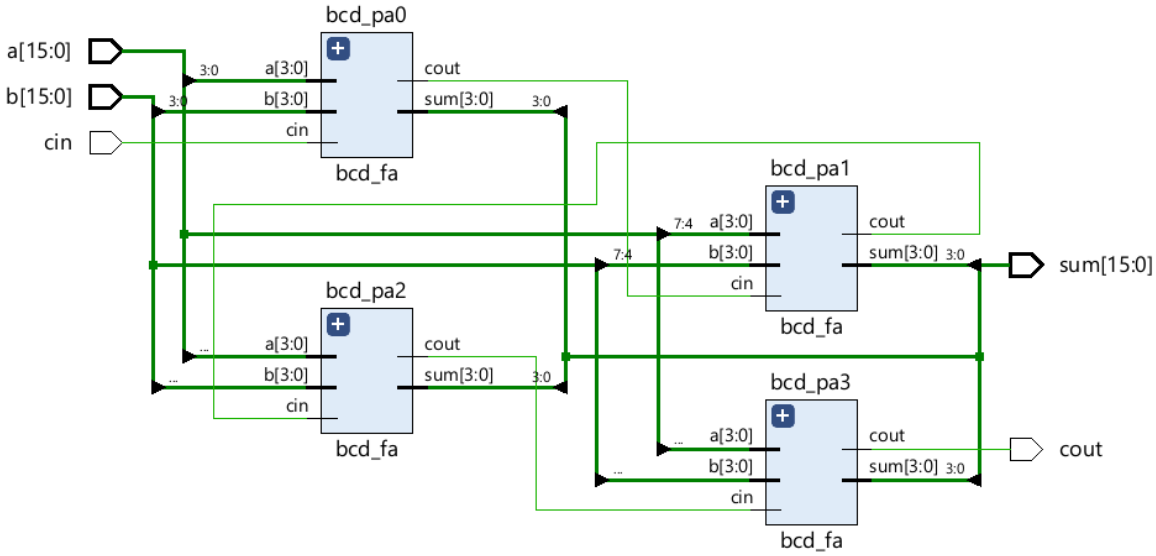
```
end structural;
```

Κώδικας VHDL – Testbench

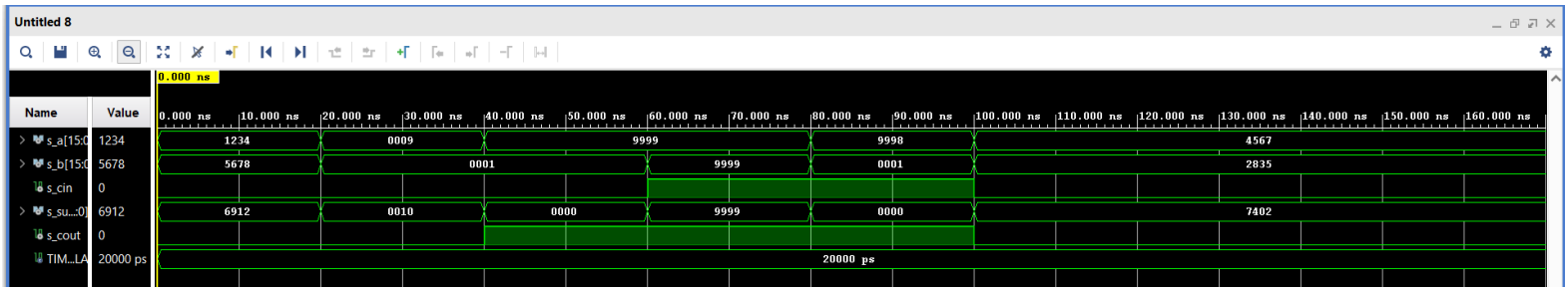
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity bcd_pa4_tb is
end bcd_pa4_tb;

architecture tb of bcd_pa4_tb is

    component bcd_pa4
        port (
            a, b : in  std_logic_vector(15 downto 0);
            cin  : in  std_logic;
            sum  : out std_logic_vector(15 downto 0);
            cout : out std_logic
        );
    end component;

    signal s_a    : std_logic_vector(15 downto 0) := (others => '0');
    signal s_b    : std_logic_vector(15 downto 0) := (others => '0');
    signal s_cin  : std_logic := '0';
    signal s_sum  : std_logic_vector(15 downto 0);
    signal s_cout : std_logic;

    constant TIME_DELAY : time := 20 ns;

begin

    DUT: bcd_pa4
        port map (
            a    => s_a,
            b    => s_b,
            cin  => s_cin,
            sum  => s_sum,
            cout => s_cout
        );

    STIMULUS: process
    begin

        s_a <= x"1234"; s_b <= x"5678"; s_cin <= '0';
        wait for (1*TIME_DELAY);

        s_a <= x"0009"; s_b <= x"0001"; s_cin <= '0';
        wait for (1*TIME_DELAY);

        s_a <= x"9999"; s_b <= x"0001"; s_cin <= '0';
        wait for (1*TIME_DELAY);

        s_a <= x"9999"; s_b <= x"9999"; s_cin <= '1';
        wait for (1*TIME_DELAY);

        s_a <= x"9998"; s_b <= x"0001"; s_cin <= '1';
        wait for (1*TIME_DELAY);

        s_a <= x"4567"; s_b <= x"2835"; s_cin <= '0';
        wait for (1*TIME_DELAY);

        wait;
    end process;


```

```
end tb;
```

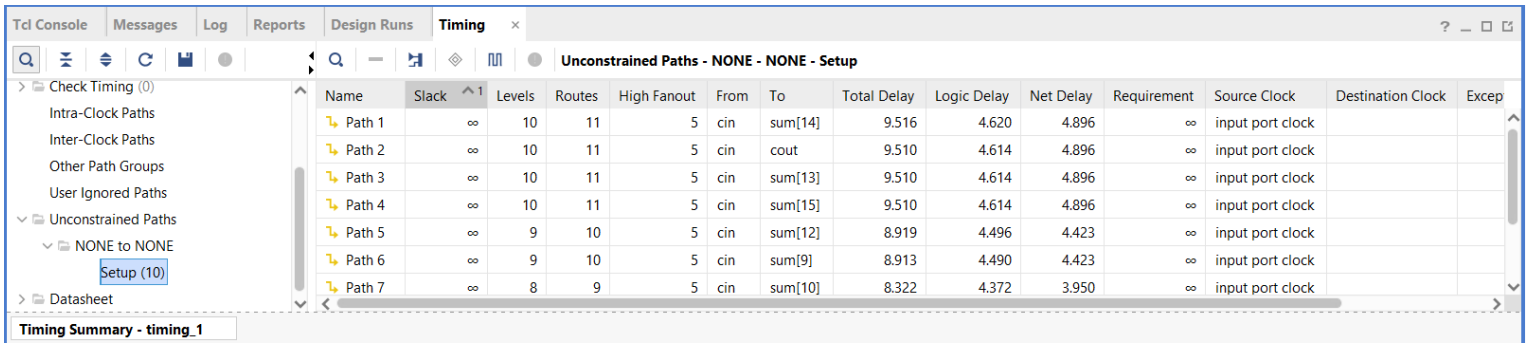
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Excep
Path 1	∞	10	11	5	cin	sum[14]	9.516	4.620	4.896	∞	input port clock		
Path 2	∞	10	11	5	cin	cout	9.510	4.614	4.896	∞	input port clock		
Path 3	∞	10	11	5	cin	sum[13]	9.510	4.614	4.896	∞	input port clock		
Path 4	∞	10	11	5	cin	sum[15]	9.510	4.614	4.896	∞	input port clock		
Path 5	∞	9	10	5	cin	sum[12]	8.919	4.496	4.423	∞	input port clock		
Path 6	∞	9	10	5	cin	sum[9]	8.913	4.490	4.423	∞	input port clock		
Path 7	∞	8	9	5	cin	sum[10]	8.322	4.372	3.950	∞	input port clock		

Σύντομη Περιγραφή Υλοποίησης

Η υλοποίηση του Parallel BCD Adder 16-bit (bcd_pa4) αποτελεί την κορύφωση της ιεραρχικής σχεδίασης στο Part B, καθώς συνδέει τέσσερις μονάδες bcd_fa σε σειρά για την πρόσθεση τετραψήφιων δεκαδικών αριθμών. Ο κώδικας ακολουθεί Structural λογική, όπου κάθε μονάδα αναλαμβάνει την πρόσθεση ενός δεκαδικού ψηφίου (4 bits) και μεταφέρει το δεκαδικό κρατούμενο στην επόμενη ανώτερη βαθμίδα μέσω του σήματος c_vector. Η RTL σχεδίαση αναπαριστά μια αλυσίδα τεσσάρων σύνθετων blocks, όπου το καθένα περιέχει εσωτερικά τη λογική διόρθωσης BCD, αναδεικνύοντας τη δύναμη της επαναχρησιμοποίησης κώδικα. Στις κυματομορφές του Vivado, αναμένουμε να δούμε την πρόσθεση μεγάλων δεκαδικών αριθμών, όπως x"1234" + x"5678", να αποδίδει το σωστό BCD αποτέλεσμα x"6912", με τα ενδιάμεσα κρατούμενα να διαδίδονται διαδοχικά από τα LSB προς τα MSB.

Το Critical Path σε αυτή την υλοποίηση φτάνει στη μέγιστη τιμή του, καθώς το κρατούμενο πρέπει να διασχίσει και τις τέσσερις βαθμίδες BCD, καθεμία από τις οποίες περιλαμβάνει δύο εσωτερικούς Parallel Adders και λογική ανίχνευσης > 9. Στο Timing Report, η συνολική καθυστέρηση (Total Propagation Delay) είναι η συσσωρευμένη καθυστέρηση όλων των επιμέρους βαθμίδων, γεγονός που καθιστά αυτό το κύκλωμα το πιο αργό της σειράς. Η ανάλυση χρονισμού επιβεβαιώνει ότι η ταχύτητα του συστήματος περιορίζεται από το "Ripple Carry" φαινόμενο, υπογραμμίζοντας την ανάγκη για πιο προηγμένες τεχνικές (όπως Carry Look-Ahead) σε συστήματα υψηλών επιδόσεων.

** Σημείωση

Ο σχολιασμός μας για το Time Analysis αφορά το critical path που προκύπτει από την κυκλωματική υλοποίηση με λογικές πύλες (AND, OR κτλ), καταχωρητές. Ωστόσο, τα αποτελέσματα που παρέχει το Vivado προκύπτουν μετά από το σταδιο Synthesis το κύκλωμα αντιμετωπίζεται σαν γενικευμένη λογική συνάρτηση, αντιστοιχίζεται

σε έναν πίνακα αληθείας και υλοποιείται με τις δομές που παρέχει το FPGA (LUTs, FFs κτλ).